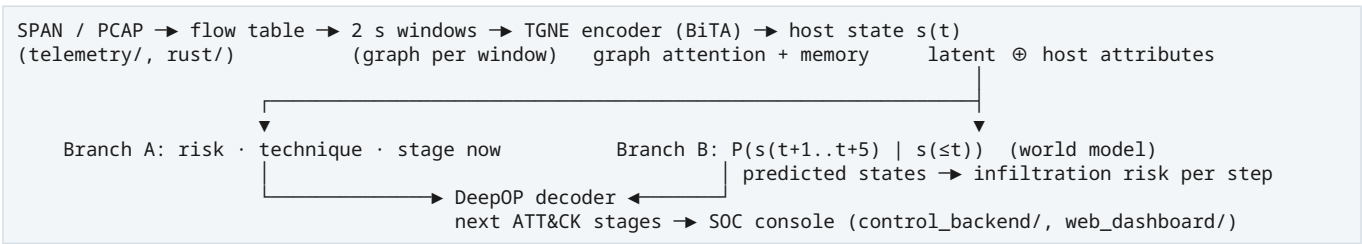


K.I.R.A. (Kinetic Intrusion Risk Anticipator) — Architecture

Problem Statement 26153 (NTRO): AI-based network attack forecasting from network traffic. K.I.R.A. learns how the state of every host in a network evolves from SPAN traffic, simulates that state forward, and reports the probability and stage of an attack *before* it completes, with the features that drive each forecast. Status: pipeline complete and tested; models in active development.

1. Pipeline



Contract	Value
Window / history / forecast	2 s windows · 15 windows (30 s) of history · 5 steps × 30 s = 150 s ahead
Edge features (per flow)	12: log fwd/bwd bytes and packets, duration, byte and packet rate, TCP/UDP/ICMP, log port, direction asymmetry; all scaled to [0, 1]
Host attributes (per window)	15 flow attributes (volumes sent/received, peers, ports, protocol mix, duration, rates, fan-out) + optional 30 packet-level (TTL, TCP window, retransmissions, inter-arrival time, payload, SYN/RST ratios, scan signatures)
Host state $s(t)$	12-D encoder latent + 15 attributes = 27-D (57-D with packet features)

2. Data and labels

- **Training:** CSE-CIC-IDS2018 from **raw PCAP** (real host addresses) plus CTU-13 botnet NetFlow. **Held-out test:** CIC-IDS2017, a different year, network and attack mix, scored once. Every capture's train/validation/test assignment is frozen in `data_unification/splits.lock.json`.
- **PCAP parsing** is a bit-exact Rust port of the Python reference (12× faster), cached per capture and keyed by the parsing code, so a code change can never reuse stale features.
- **Labels.** CIC-2018 publishes attack *time windows*. A flow is labelled with the attack only if its {source, destination} pair appears on the attack rows; windows that name no participants are UNKNOWN and excluded from supervision, never stamped onto every host active at that moment.
- **Leakage guards:** nodes are scoped per capture; train precedes validation precedes test inside every capture; the encoder never sees validation or test captures; no label reaches serving.

3. Models

TGNE encoder (BiTA, `bita/`). A temporal graph network: for each host, graph attention (2 heads) over its 10 most recent interactions, plus a 12-D memory updated by BiTA's BiGRU-Transformer aggregator over the messages each host received, so context older than the window persists. Self-supervised by temporal link prediction, with an auxiliary flow-category head (focal loss, inverse-frequency class weights). Model selection: harmonic mean of link-prediction AP on *unseen* hosts and category macro-F1, so neither can hide a collapse of the other.

Branch A (GNN-LSTM, `branch_a_gnn_lstm/`). LSTM (1 × 256) over the last 15 host states. Three heads: infiltration risk, ATT&CK technique (14 classes), and kill-chain stage. Loss $0.5 \cdot \text{risk} + 0.3 \cdot \text{technique} + 0.2 \cdot \text{stage}$. Post-hoc temperature scaling and a split-conformal interval on the risk; the alert threshold maximises F1 within an alert budget of 2× the base rate (fitted on validation only).

Branch B: world model (`branch_b_world_model/`). A causal transformer (3 layers, 4 heads) over the history with continuous-time encoding of the real gaps between windows. It predicts the next state as a residual and a log-variance, giving a **Gaussian predictive distribution** per step, and rolls out autoregressively for 5 steps. A risk head reads each predicted state. It must beat the copy-the-last-state baseline by 2% before DeepOP is trained on it.

DeepOP (`deepop_decoder/`). An encoder over the observed technique sequence (Branch A) and a decoder with cascaded window attention over Branch B's predicted states; it emits the next ATT&CK tokens (Recon, Credential Access, Initial Access, C2, Exfiltration, Impact, Benign). It is trained on Branch B's own rollouts with corrupted observed history (10% dropped, 10% substituted), the conditions it meets in production.

4. Training targets and procedure

- **Risk = hazard of onset:** $\exp(-\Delta t / \tau)$, with Δt the time to the host's next attack window and $\tau = 150$ s, the forecast horizon. Targets whose future falls in unobserved traffic (dropped UNKNOWN spans, the end of a capture) are **censored**, not counted as safe.
- Encoder batches stay time-ordered (memory requires it) but interleave all captures by within-capture progress, so each update sees every attack type; each host's own sequence is unchanged.
- One **training guard** for all models: warm-up, gradient clipping, non-finite steps skipped, roll-back to the best epoch at half the learning rate, early stopping, crash-safe resume.
- **Speed:** Rust parsing and neighbour sampling, columnar caches, a batch planner in a separate process, and whole training steps replayed as CUDA graphs. Results are identical to the reference implementation (bit for bit, or within fp32 rounding).

5. Evaluation

- **Baselines on identical inputs and targets:** logistic regression (the problem statement's benchmark) and persistence. Reported: precision, recall, F1, **false-positive rate** and AUC.
- **Early warning** is scored separately: hosts benign in their last observed window. Continuations of an ongoing attack are trivial (persistence gets them right), so headline numbers never mix the two. DeepOP is likewise scored on stage *transitions*, which persistence cannot predict.
- Macro-F1 over present classes, never accuracy (the traffic is >80% benign); classes absent from training are reported separately on the cross-year test; three seeds, mean and spread.

6. Explainability and serving

- **Driving features:** Input $\times$ Gradient attribution of the risk over every input window and feature, grouped for analysts (TCP flags, scan signature, timing, volume, protocol, encoder latent), and which windows drove the forecast.
- **Serving** (`control_backend/`, FastAPI + WebSocket): passive AF_PACKET capture on the SPAN NIC, the same window grid, padding, feature code and time encoding as training (pinned by parity tests), a discovery-only topology, per-step risk with conformal bands, the forecast tree of the top-3 DeepOP continuations, and heuristic campaign correlation (labelled as such). The verdict shown is always the model's; analyst actions never feed back into scoring. Runs fully offline.

7. Problem-statement coverage

Requirement	Where it is met
Flow- and packet-level features	12 edge + 15 host flow features; 30 packet-level host features (Rust PCAP parser)
State as feature vector / graph	interaction graph per 2 s window $\rightarrow$ TGNE host state $s(t)$
Learn $P(S(t+1) \mid S(t))$	Branch B: Gaussian predictive distribution, causal transformer (+ GNN encoder, LSTM)
K-step forward simulation	5-step autoregressive rollout over 150 s
Infiltration probability over time	per-step hazard risk with conformal bands
MITRE ATT&CK stage	Branch A (current stage), DeepOP (next stages along the rollout)
Driving features	Input $\times$ Gradient attribution, grouped by flags / ports / scans / timing / volume
Generalise to unseen attacks	cross-year test (train 2018 $\rightarrow$ test 2017), unseen classes reported separately
Logistic-regression benchmark	same inputs and target; F1, precision, recall, FPR, AUC
Offline interface, PCAP input	SOC console, PCAP replay through the sensor path, no cloud dependency

8. Status and next steps

The pipeline, console and 1,296 tests are complete. Model retraining on the corrected labelling is in progress. Next: per-host attribution of the remaining unlabelled CIC-2018 days, PCAP-derived test flows to evaluate packet-level features, and lateral-movement data.